

基于 Verilog 的 I2C 总线控制器设计与验证

本文为本科毕业论文初稿。封面、任务书、姓名、学号、学院、专业、指导教师、日期等信息请在最终排版阶段按学校模板补充。本文中的项目事实主要依据 `test/project-report.md` 与 项目/`verilog-i2c-master` 的 README、RTL 源码和测试平台文件整理；未实际运行仿真或综合的部分均以“待补充”标明。

摘要

I2C 总线因连线简单、设备扩展方便、协议开销较低，被广泛用于 FPGA 板级外设配置、传感器访问、EEPROM 读写、时钟芯片初始化和片上系统低速控制通道。针对 FPGA 系统中 I2C 外设访问对可复用硬件 IP 的需求，本文以 Verilog I2C interface 项目为研究对象，设计并分析一种基于 Verilog HDL 的 I2C 总线控制器及其验证方法。该设计以 I2C 主机核心和 I2C 从机核心为基础，采用 AXI Stream 风格接口组织命令流和数据流，并通过 AXI Lite、Wishbone 等总线封装方式实现与处理器或片上总线系统的连接。同时，系统提供基于 ROM 表的 I2C 初始化模块，可用于无通用处理器参与的上电外设配置。

本文首先分析 I2C 协议的起始条件、停止条件、地址传输、读写控制、ACK/NACK 应答、开漏总线和时钟拉伸等关键机制；随后从需求分析、总体结构、模块划分、状态机设计、接口设计和寄存器映射等方面展开系统设计。主机模块采用命令层状态机与位级物理层状态机分层实现，完成 start、repeated start、读位、写位和 stop 等操作；从机模块通过地址匹配、输入滤波、AXI Stream 数据转换和时钟拉伸机制实现 I2C 从设备功能；AXI Lite 与 Wishbone 封装模块通过状态寄存器、命令寄存器、数据寄存器和分频寄存器完成上层控制。最后，本文结合 MyHDL 与 Icarus Verilog 协同仿真平台，给出测试思路、验证场景和结果记录方式。

研究表明，该类 I2C 控制器具有模块化程度高、接口适配性强、可综合性好和便于 FPGA 系统集成等特点。由于当前材料尚未包含本机仿真日志、综合报告和板级实测数据，本文将相关实测结果列为后续补充项。整体而言，本文工作可为 FPGA 中低速外设控制器设计、标准总线封装和硬件模块验证提供参考。

关键词：I2C 总线，Verilog，FPGA，AXI Lite，Wishbone，协同仿真

Abstract

The I2C bus is widely used in FPGA-based peripheral configuration, sensor access, EEPROM read and write operations, clock chip initialization, and low-speed control channels in system-on-chip designs because of its simple wiring, flexible device expansion, and low protocol overhead. To meet the demand for reusable hardware IP for I2C peripheral access in FPGA systems, this thesis studies a Verilog-based I2C bus controller and its verification method based on the Verilog I2C interface project. The design is built around an I2C master core and an I2C slave core. AXI Stream-style interfaces are used to organize command and data streams, while AXI Lite and Wishbone wrappers are provided

to connect the controller to processors or on-chip bus systems. In addition, an ROM-table-driven I2C initialization module is provided for power-on peripheral configuration without the participation of a general-purpose processor.

This thesis first analyzes key mechanisms of the I2C protocol, including start and stop conditions, address transmission, read/write control, ACK/NACK responses, open-drain bus behavior, and clock stretching. Then the system is discussed from the perspectives of requirement analysis, overall architecture, module partitioning, state machine design, interface design, and register mapping. The master module adopts a layered structure consisting of a command-level state machine and a bit-level physical state machine to implement start, repeated start, bit read, bit write, and stop operations. The slave module implements I2C slave functionality through address matching, input filtering, AXI Stream data conversion, and clock stretching. The AXI Lite and Wishbone wrapper modules expose status, command, data, and prescale registers for upper-layer control. Finally, a co-simulation verification platform based on MyHDL and Icarus Verilog is introduced, including test scenarios and result recording methods.

The analysis shows that this type of I2C controller has high modularity, strong interface adaptability, good synthesizability, and convenient integration into FPGA systems. Since the current materials do not include local simulation logs, synthesis reports, or board-level measurement data, these experimental results are marked as items to be supplemented in future work. Overall, this thesis can serve as a reference for FPGA low-speed peripheral controller design, standard bus wrapping, and hardware module verification.

Key words: I2C bus, Verilog, FPGA, AXI Lite, Wishbone, co-simulation

1 绪论

1.1 研究背景与意义

在嵌入式设备、工业控制板卡以及各类智能硬件中，主控芯片周围通常会挂接较多低速外设，例如 EEPROM、RTC、温湿度传感器、显示控制芯片、时钟发生器、电源管理器件和板级监控芯片等。这些器件的数据吞吐量要求并不高，但对连线数量、接口复杂度和控制流程有较强约束。I2C 总线只依赖 SCL 与 SDA 两根信号线即可完成多设备寻址和读写控制，因此在板级低速外设管理中具有较高使用频率。

I2C 总线由 SCL 和 SDA 两根信号线组成，可通过 7 位或 10 位地址区分多个从设备。由于其采用开漏输出和上拉电阻结构，多个器件可以共享同一总线，并通过起始条件、停止条件、地址字段、读写位和应答位完成通信控制。与 SPI 相比，I2C 在连接多个低速外设时可以节省片选线；与 UART 相比，I2C 支持总线寻址和主从设备结构，更适合板级多外设管理。因此，在 FPGA 系统中实现一个稳定、可复用、易集成的 I2C 控制器具有实际工程意义。

在 FPGA 系统中，实现 I2C 访问通常有两种思路：一类是由软核或硬核处理器运行驱动程序，再通过外设控制器访问总线；另一类是直接使用 RTL 逻辑完成时序

控制。前一种方式便于软件配置和调试，后一种方式更适合上电早期初始化、低延迟响应或不希望引入处理器依赖的场合。若将硬件控制器进一步包装为 AXI Lite 或 Wishbone 外设，处理器仍可通过寄存器参与控制，纯硬件逻辑与 SoC 集成两种使用方式便能够在同一 IP 结构中兼容。

本文研究的 Verilog I2C interface 项目提供 I2C 主机、I2C 从机、AXI Lite 封装、Wishbone 封装、AXI Stream FIFO 和上电初始化模板等模块，具有较完整的硬件 IP 结构。围绕该项目展开毕业论文写作，可以系统分析 I2C 协议硬件实现、标准片上总线封装、状态机设计、流式接口设计和协同仿真验证方法，有助于加深对数字系统设计流程的理解。

1.2 国内外研究现状

I2C 协议起源于 Philips 公司提出的板级串行通信方案，后续由 NXP 持续维护并发布规范文档。UM10204 手册对两线连接方式、传输格式、应答机制、仲裁过程、时钟同步以及不同速率等级进行了说明。本文在分析控制器实现时，主要参考其中关于起始/停止条件、SDA 采样窗口、SCL 拉伸和多器件共享总线的规定，以保证 RTL 行为与协议要求相一致。

在实际工程中，I2C 控制器通常不会孤立存在，而是以可复用 IP 的形式接入 FPGA 或 SoC 系统。AMD/Xilinx 的 AXI IIC Bus Interface IP 采用 AXI 外设接口，便于处理器通过寄存器完成配置与传输控制；OpenCores 的 I2C controller core 则面向 Wishbone 总线，体现了开源 SoC 生态中的外设封装方式。由此可见，控制器设计除了满足 I2C 时序，还需要考虑上层总线、寄存器模型和验证平台的配合。

在国内研究中，围绕“基于 FPGA 的 I2C 总线控制器设计”“基于 Verilog 的 I2C 协议实现”“I2C 总线接口设计与仿真”等方向已有较多课程设计、工程论文和学位论文。相关工作通常关注 I2C 总线时序、有限状态机划分、FPGA 实现、ModelSim 仿真和外设读写验证。与单一 I2C 主机控制器相比，本文所分析的项目具有主从双向支持、多种片上总线封装和 MyHDL 协同仿真环境等特点，因此更适合作为完整硬件 IP 设计与验证案例。

综合相关资料可以看出，本文讨论的重点并不是提出新的 I2C 协议，而是围绕 FPGA 场景梳理一个控制器 IP 如何做到结构清晰、便于综合、易于复用并具备可验证性。因此，后文将以模块化设计为主线，分别分析主机核心、从机核心、总线接口封装以及测试平台之间的关系。

1.3 研究内容与技术路线

本文主要研究内容包括以下几个方面：

1. I2C 协议原理分析。梳理 I2C 总线的物理连接方式、起止条件、地址传输、读写方向、ACK/NACK 应答、时钟拉伸和总线状态判断，为硬件实现提供协议基础。
2. 系统需求分析。根据 Verilog I2C interface 项目资料，总结系统应支持的主机通信、从机通信、AXI Lite 封装、Wishbone 封装、初始化序列和仿真验证等功能。

3. 总体结构设计。分析项目中 RTL 与测试平台的目录结构，给出 I2C 核心层、流式接口层、总线封装层和验证层之间的关系。
4. 关键模块设计。重点分析 `i2c_master`、`i2c_slave`、`i2c_master_axil`、`i2c_master_wbs_8`、`i2c_init` 和 `axis_fifo` 等模块的功能、接口、状态机和数据流。
5. 仿真验证方法。结合 MyHDL 和 Icarus Verilog 协同仿真环境，讨论主机读写、从机读写、FIFO 缓冲、寄存器访问、ACK 检测、时钟拉伸等测试场景。
6. 总结不足与改进方向。针对当前材料中缺少综合结果、板级验证和完整测试日志的问题，提出后续完善方案。

本文的研究路线可以概括为四个步骤。首先，根据 I2C 协议文档和项目 README 明确控制器应具备的功能边界；其次，结合项目报告与主要 RTL 文件梳理模块层次、接口关系和数据流向；再次，围绕主从控制器状态机、片上总线寄存器和初始化逻辑展开实现分析；最后，根据现有测试平台设计验证场景，并在总结中说明当前材料不足和后续补充方向。

1.4 论文组织结构

全文按照“背景分析—理论基础—需求设计—实现验证—总结展望”的顺序展开。第 1 章说明课题背景、研究意义、已有工作和本文内容安排；第 2 章介绍 I2C、Verilog HDL、FPGA、AXI Stream、AXI Lite、Wishbone 以及协同仿真等基础知识；第 3 章结合项目资料归纳系统需求和设计约束；第 4 章说明总体架构、模块划分、接口和数据流；第 5 章分析主机、从机、总线封装、初始化模块和 FIFO 的实现；第 6 章给出测试场景与结果记录要求；第 7 章对全文工作进行归纳，并提出后续完善方向。

2 相关技术与理论基础

2.1 I2C 总线协议

I2C 是一种以时钟同步为基础的串行总线，通信时主要使用 SCL 与 SDA 两根线。总线空闲时，外部上拉电阻使两根线保持高电平。主设备若要开始一次访问，需要在 SCL 保持高电平时将 SDA 从高电平拉低；若要结束访问，则在 SCL 为高电平时释放 SDA，使其由低电平回到高电平。前者对应起始条件，后者对应停止条件。

I2C 传输通常以 8 位为单位进行。主设备首先发送 7 位从设备地址和 1 位读写方向位，其中读写位为 0 表示写操作，为 1 表示读操作。每发送 8 位数据后，接收方需要在第 9 个时钟周期给出 ACK 或 NACK。ACK 通常表现为接收方拉低 SDA，NACK 则表现为释放 SDA 保持高电平。通过 ACK/NACK，发送方可以判断对方是否正确接收数据，或判断读操作是否需要结束。

I2C 的一个重要特性是开漏总线结构。设备不能主动驱动高电平，只能拉低总线或释放总线。因此，当多个设备共享总线时，只要任一设备拉低信号线，总线就表现为低电平。这一特性支持多设备共享、仲裁和时钟拉伸。时钟拉伸是指从设备在无法及时提供或接收数据时，可以拉低 SCL 暂停传输，直到准备好后释放 SCL。

从 RTL 实现角度看，控制器既要判断 start、stop、SCL 边沿、SDA 边沿和

ACK/NACK, 又不能把输出逻辑简单等同于引脚实际电平。原因在于 I2C 采用开漏连接, 总线电平可能受到其他器件或时钟拉伸行为影响。为此, 项目将信号拆分为 `scl_i/scl_o/scl_t` 与 `sda_i/sda_o/sda_t`, 分别承担输入采样、输出值和三态控制功能, 使总线释放与拉低动作能够被清晰表达。

2.2 Verilog HDL 与 FPGA 设计

Verilog HDL 面向硬件结构描述, 可用于表达组合逻辑、时序寄存器、有限状态机以及模块之间的连接关系。其代码在综合后并不是按软件语句顺序执行, 而是被映射到 FPGA 内部的查找表、触发器、存储资源和布线网络中。借助 FPGA 提供的可编程逻辑、片上 RAM、时钟管理和 I/O 资源, 设计者可以在硬件平台上快速实现和调试数字系统。

FPGA 工程通常要经历 RTL 设计、功能仿真、综合实现、布局布线、时序约束检查以及实物验证等步骤。对于通信接口而言, 控制逻辑既要关注每一位在总线上的变化, 也要管理字节级数据和上层握手信号。若将这些逻辑混在一起, 状态转换会变得复杂。因此, 本课题中的 I2C 控制器更适合把位级时序和事务级控制分开组织。

本文项目的 RTL 源码采用 Verilog 2001 风格, 主要模块均以可综合形式编写。模块中大量使用寄存器保存状态、地址、数据、计数器和握手标志, 并通过组合逻辑计算下一状态, 通过时钟边沿更新寄存器。这种写法符合常见 FPGA 综合工具的处理方式。

2.3 AXI Stream 与 AXI Lite 接口

AXI 属于 AMBA 规范体系中的常用接口。与面向寄存器读写的总线不同, AXI Stream 更适合传递连续数据。其握手机制由 `tvalid` 和 `tready` 共同决定: 发送侧声明数据有效, 接收侧声明可以接收, 二者同时成立时数据才在当前时钟周期完成交接。`tdata` 用于携带数据内容, `tlast` 通常用来标记帧结束或事务边界。

在本文分析的项目中, I2C 主机命令、写入数据和读出数据都采用类似 AXI Stream 的握手方式组织。这样做可以把 I2C 位级时序与上层数据缓冲分开处理, 也便于在控制器前后接入 FIFO 或其他流式模块。以多字节访问为例, 写数据侧可利用 `tlast` 指示最后一个字节, 读数据侧则通过 `m_axis_data_tlast` 表示当前读事务已经结束。

AXI Lite 可理解为面向控制寄存器访问的轻量化 AXI 接口, 常用于处理器配置低速外设。该接口由写地址、写数据、写响应、读地址和读数据等通道组成, 传输带宽要求不高, 但访问流程规范。项目中的 `i2c_master_axil` 便利用这一接口把 I2C 主机包装成 AXI Lite 从设备, 使软件能够通过状态、命令、数据和分频等寄存器完成控制。

2.4 Wishbone 总线接口

Wishbone 是开源硬件项目中较常见的片上总线接口, 许多软核处理器和开源 SoC 会采用它连接外设。其基本信号包括地址、写数据、读数据、写使能、周期、选通和应答等。对于 I2C 控制器这类低带宽寄存器外设而言, Wishbone 与 AXI Lite 一样, 都可以承担处理器与控制寄存器之间的访问通道。

本文项目提供 `i2c_master_wbs_8` 和 `i2c_master_wbs_16` 两种 Wishbone 主机封装，分别适配 8 位和 16 位数据宽度。以 8 位封装为例，寄存器包括 Status、FIFO Status、Cmd Addr、Command、Data、Prescale Low 和 Prescale High 等。通过 Wishbone 封装，I2C 控制器可以集成到采用 Wishbone 总线的开源 SoC 系统中。

2.5 MyHDL 与 Icarus Verilog 协同仿真

硬件模块完成编码后，不能只依赖语法检查判断其正确性，还需要通过仿真观察功能行为是否满足预期。Icarus Verilog 可以编译 Verilog 测试平台与被测设计，生成可运行的仿真文件；MyHDL 则利用 Python 描述测试激励、行为模型和总线交互逻辑。两者结合后，既能运行真实 RTL，又能用 Python 编写更灵活的验证环境。

本文项目的测试平台采用 MyHDL 与 Icarus Verilog 协同仿真方式。Python 测试脚本中创建 AXI Stream source/sink、I2C 存储器模型、AXI Lite BFM 和 Wishbone 模型，再通过 Icarus Verilog 编译 Verilog DUT，并使用 `vvp -m myhdl` 加载 MyHDL VPI 实现联动。该方法既能利用 Verilog RTL 的真实实现，又能利用 Python 语言编写灵活测试激励，有利于提高验证效率。

3 系统需求分析

3.1 设计目标

本文设计目标是实现并分析一种可复用的 I2C 总线控制器硬件 IP，使其能够在 FPGA 或 SoC 系统中完成 I2C 外设访问。结合项目资料，可将目标细化为以下几点：

1. 实现 I2C 主机控制器，支持基本读写、多字节写、起始条件、重复起始条件和停止条件。
2. 实现 I2C 从机控制器，支持地址匹配、读写方向判断、数据收发和总线状态输出。
3. 提供 AXI Stream 风格接口，便于命令和数据流与 FIFO 或上层逻辑连接。
4. 提供 AXI Lite 封装，使处理器能够通过寄存器控制 I2C 主机。
5. 提供 Wishbone 封装，使控制器适配开源 SoC 或 Wishbone 总线系统。
6. 提供 I2C 初始化模块，使 FPGA 可在上电后自动配置一个或多个 I2C 外设。
7. 提供可运行的仿真测试平台，支持对主机、从机和总线封装模块进行功能验证。

3.2 功能需求

3.2.1 I2C 主机功能需求

I2C 主机模块需要根据上层命令驱动 SCL 和 SDA，完成总线传输。其功能需求包括：

- 接收 7 位目标地址；
- 支持 read、write、write multiple 和 stop 命令；
- 支持 start 和 repeated start；

- 支持多字节写入，并通过 `tlast` 标记结束；
- 支持读数据输出，并在需要时标记最后一个字节；
- 检测从机 ACK，未检测到 ACK 时输出 `missed_ack` 状态；
- 输出 `busy`、`bus_control` 和 `bus_active` 等状态；
- 通过 `prescale` 配置 I2C 时钟速率。

3.2.2 I2C 从机功能需求

I2C 从机模块需要响应外部主机访问，并将 I2C 总线事务转换为内部流式数据。其功能需求包括：

- 支持 7 位设备地址配置；
- 支持地址掩码配置，以实现单地址或部分地址匹配；
- 检测 I2C 起始条件和停止条件；
- 支持外部主机写入数据，并将数据输出到 AXI Stream 接口；
- 支持外部主机读取数据，并从 AXI Stream 输入接口获取待发送字节；
- 在数据未准备好时支持 SCL 时钟拉伸；
- 输出当前总线地址、是否被寻址、总线是否活动和模块是否忙碌等状态。

3.2.3 总线封装功能需求

AXI Lite 和 Wishbone 封装模块用于将 I2C 控制器接入上层系统。封装模块应满足以下需求：

- 将命令、数据、分频和状态映射为寄存器；
- 提供 FIFO 空、满、溢出等状态；
- 支持软件或总线主设备写入命令和数据；
- 支持软件或总线主设备读取接收数据和错误状态；
- 保持寄存器访问与 I2C 总线传输之间的时序解耦。

3.2.4 初始化功能需求

`i2c_init` 模块面向上电自动配置场景，需求包括：

- 使用内部 ROM 表保存初始化命令；
- 支持单设备初始化；
- 支持多个设备执行相同初始化序列；
- 支持写地址、写数据、延时和停止命令；
- 输出符合 I2C 主机命令接口的数据流。

3.3 非功能需求

1. 可综合性：核心 RTL 应采用常见综合工具能够识别的写法，避免仅适用于仿真的结构。
2. 可复用性：模块端口和参数应保持清晰，便于迁移到其他 FPGA 工程。
3. 可配置性：I2C 分频值、FIFO 深度和地址掩码等参数应根据系统需求调整。
4. 可验证性：工程应配套测试脚本和行为模型，支持自动化功能仿真。

5. 接口兼容性：设计需要同时兼顾 AXI Stream、AXI Lite 和 Wishbone 等连接方式。
6. 通信可靠性：控制逻辑应正确响应 ACK/NACK、总线忙闲、时钟拉伸和 FIFO 状态变化。

3.4 设计约束

本文项目存在以下约束：

1. 工程源码采用 Verilog 2001，论文分析和后续修改都应保持可综合 RTL 风格。
2. SCL 与 SDA 遵循开漏总线特性，顶层连接时需要正确处理三态释放和低电平驱动。
3. `prescale` 与系统时钟、目标 I2C 速率直接相关，配置时不能脱离实际时钟条件。
4. 若运行协同仿真，需要提前准备 Python、MyHDL、Icarus Verilog 以及 `myhdl.vpi` 环境。
5. 目前材料没有提供完整仿真日志、综合报告和板级截图，论文中相关结果只能标注为待补充。

4 系统设计

4.1 总体架构

结合 RTL 目录和测试平台文件，可以把该设计理解为由内到外的四个部分。最内层是直接控制 SCL/SDA 的 I2C 主从协议核心；第二层使用流式接口和 FIFO 传递命令及数据；第三层通过 AXI Lite、Wishbone 等封装接入处理器或片上互连；最外层则由 Python 行为模型和 Verilog 测试顶层共同构成验证环境。

协议核心层由 `i2c_master` 与 `i2c_slave` 构成。前者作为总线主设备主动产生起始条件、地址字段、读写控制和停止条件；后者作为从设备监听总线，在地址匹配后完成数据接收或发送。两个核心模块都直接面对 SCL/SDA 相关信号，因此需要处理 ACK/NACK、总线忙闲状态以及开漏接口控制。

流式接口层主要由 AXI Stream 风格接口和 `axis_fifo` 组成。主机命令、写数据、读数据以及从机数据收发均可通过 `ready/valid/tlast` 握手机制组织。FIFO 模块用于缓冲命令和数据，降低上层访问与底层总线时序之间的耦合。

总线封装层的作用是把底层流式命令和数据接口转换为处理器更容易使用的寄存器模型。AXI Lite 与 Wishbone 封装模块分别面向不同片上总线生态，上层软件只需要访问状态、命令、数据和分频等寄存器，就可以间接控制 I2C 传输过程。

验证相关文件集中在 `tb/` 目录。Python 部分提供 I2C 行为模型、AXI Stream 端点、AXI Lite BFM 和 Wishbone 模型，Verilog 部分则给出被测模块连接和仿真顶层。测试脚本负责产生读写激励、驱动 DUT 运行、检查返回数据，并在需要时输出波形供后续分析。

4.2 模块划分

系统主要模块划分如表 4.1 所示。

表 4.1 系统主要模块划分

模块	文件	功能
AXI Stream FIFO	<code>axis_fifo.v</code>	提供参数化流式数据缓冲
I2C 初始化模块	<code>i2c_init.v</code>	解释 ROM 初始化表并输出 I2C 命令
I2C 主机核心	<code>i2c_master.v</code>	生成 I2C 主机读写时序
I2C 主机 AXI Lite 封装	<code>i2c_master_axil.v</code>	提供 32 位 AXI Lite 寄存器接口
I2C 主机 Wishbone 封装	<code>i2c_master_wbs_8.v</code> 、 <code>i2c_master_wbs_16.v</code>	提供 8 位和 16 位 Wishbone 接口
I2C 从机核心	<code>i2c_slave.v</code>	将 I2C 从机事务转换为 AXI Stream 数据
I2C 从机 AXI Lite master 封装	<code>i2c_slave_axil_master.v</code>	将 I2C 从机访问映射为 AXI Lite master 操作
I2C 从机 Wishbone master 封装	<code>i2c_slave_wbm.v</code>	将 I2C 从机访问映射为 Wishbone master 操作

4.3 数据流设计

主机写数据流为：上层逻辑或处理器写入命令和数据，命令进入主机命令接口，数据进入写数据接口；主机模块产生 I2C 起始条件，发送地址和写方向位，随后逐字节发送数据并检测 ACK；若命令要求停止，则发送停止条件。

主机读数据流为：上层写入读命令，主机模块产生起始条件并发送地址和读方向位；从机应答后，主机逐位采样 SDA，组成 8 位数据，通过读数据输出接口返回上层；若为最后一个字节，则产生 NACK 或 stop，并通过 `tlast` 标记。

从机写数据流为：外部主机发送地址和写方向位，从机地址匹配后接收后续字节；接收的数据被转换为 AXI Stream 输出。为了判断最后一个写入字节，从机模块对输出数据进行一个字节时间的延迟，以便结合 stop 条件设置 `tlast`。

从机读数据流为：外部主机发送地址和读方向位，从机地址匹配后从 AXI Stream 输入侧读取待发送数据；若本地数据未准备好，从机拉低 SCL 进行时钟拉伸；数据有效后，从机逐位驱动 SDA 发送给外部主机。

4.4 接口设计

4.4.1 I2C 接口

I2C 相关端口没有采用单一输入输出信号，而是把采样、输出值和三态控制拆开。以主机模块为例，`scl_i` 和 `sda_i` 用于读取总线实际电平，`scl_o` 和 `sda_o` 表示模块希望输出的值，`scl_t` 和 `sda_t` 用于控制是否释放引脚。在 FPGA 顶层连接

时，释放总线意味着引脚处于高阻态并依靠上拉电阻变为高电平；需要发送低电平时，控制器再主动拉低对应信号线。

4.4.2 命令接口

主机命令接口并不直接传入完整波形，而是用若干字段描述一次事务的控制意图，包括目标地址、起始条件、读写方向、连续写入和停止条件等。命令传递遵循 valid/ready 规则，只有发送方声明有效且控制器处于可接收状态时才会采纳该命令。借助这些控制位，上层可组合出单字节访问、多字节写入和 repeated start 等场景。

4.4.3 数据接口

数据接口按 8 位字节组织，写通道和读通道都使用 `tdata` 传递数据，并用 `tvalid`、`tready` 完成握手确认。多字节事务中，`tlast` 用来指出当前字节是否为本次传输的末尾。采用这种流式格式后，控制器前后可以较自然地连接 FIFO 或自定义控制状态机，测试脚本也能方便地注入和接收数据。

4.4.4 寄存器接口

在 AXI Lite 封装中，I2C 主机被抽象成若干可读写寄存器。软件读取状态寄存器即可了解模块忙闲、总线控制权、ACK 缺失和 FIFO 状态；写命令寄存器可提交地址以及 start、read、write、stop 等控制信息；数据寄存器用于发送或接收字节；分频寄存器则设置 SCL 生成参数。这样，上层软件主要面对寄存器读写，而不用直接参与底层流式握手。

5 系统实现

5.1 I2C 主机模块实现

`i2c_master` 是整个控制器中承担主动传输任务的核心模块。该模块没有把所有控制逻辑集中在一个状态机中，而是把事务处理与位级时序分开：上层命令控制逻辑负责解析 read、write、write multiple、stop 等输入，并判断是否需要 start 或 repeated start；底层物理控制逻辑负责按照 I2C 规范驱动 SCL 与 SDA，完成逐位发送、采样和停止条件生成。

主机模块定义了多个命令层状态，包括空闲、活动写、活动读、起始等待、起始、地址发送、写数据、读数据和停止等状态。空闲状态下，模块等待上层命令；当收到读写命令后，模块根据总线活动状态和地址变化判断是否需要发送起始条件；进入地址发送状态后，模块发送 7 位地址和读写位；随后根据命令类型进入读或写状态。

物理层状态机主要负责把抽象命令转换成 SCL/SDA 上的具体动作。发送数据位时，控制器先在 SCL 低电平阶段稳定 SDA，再拉高 SCL 供从设备采样，之后重新拉低 SCL 进入下一位。读取数据位时，主机释放 SDA，由从设备驱动数据线，并在 SCL 高电平窗口内采样。对于 stop 条件，控制器需要保证 SDA 的低到高变化发生在 SCL 为高电平期间。

主机模块还通过 `prescale` 控制 SCL 时钟周期。源码注释给出的计算方式为：

$\text{prescale} = \text{Fclk} / (\text{FI2Cclk} * 4)$

式中 Fclk 表示输入到控制器的系统时钟， FI2Cclk 表示期望得到的 I2C 总线时钟。模块内部依据该分频值推进物理层状态机，使同一 RTL 在不同系统时钟条件下仍可配置出合适的 SCL 频率。

5.2 I2C 从机模块实现

i2c_slave 与主机模块不同，它不能主动决定事务何时开始，而是持续监听外部主机在总线上的操作。SCL 和 SDA 均来自芯片引脚，可能受到毛刺或边沿抖动影响，因此源码中设置 FILTER_LEN 对输入进行滤波。滤波后的相邻采样值用于识别 SCL、SDA 的边沿变化，进而判断 start、stop 以及每一位数据的采样窗口。

检测到 start 后，从机开始接收地址字段，依次采样 7 位设备地址和 1 位读写方向位。地址接收完成后，模块将总线上的地址与 device_address 按 $\text{device_address_mask}$ 进行比较。匹配成功时，从机拉低 SDA 返回 ACK，并根据方向位进入读数据或写数据流程；匹配失败时，模块不参与后续数据阶段，而是释放总线等待新的事务。

在写方向下，外部主机向从机发送数据，从机逐位接收并组成字节。由于只有在下一个字节或 stop 条件出现时才能确定前一个字节是否为最后一个字节，因此模块将 I2C 写入字节延迟一个字节时间后输出到 AXI Stream 接口。这样可以更准确地设置 m_axis_data_tlast 。

当外部主机执行读操作时，从机需要从本地 AXI Stream 输入侧取得待发送字节，再逐位送到 SDA。若上层暂时没有给出有效数据，模块不会随意输出无效内容，而是保持 SCL 为低电平，使主机时钟被暂停。待输入数据到达后，从机释放 SCL 并继续传输，这就是项目中对时钟拉伸机制的具体使用。

5.3 AXI Lite 封装模块实现

i2c_master_axil 的作用是为 I2C 主机增加 AXI Lite 寄存器访问层，使其可以作为处理器系统中的普通外设使用。软件通过 AXI Lite 写入命令和数据，或读取状态与返回数据，而模块内部再把这些寄存器访问转换为主机核心所需的流式接口。源码中还提供默认分频、固定分频开关以及命令、写数据、读数据 FIFO 深度等参数，用于适配不同系统需求。

该模块的寄存器映射包括：

表 5.1 AXI Lite 封装寄存器

地址	寄存器	功能
0x00	Status	busy、bus_cont、bus_act、miss_ack、FIFO 状态
0x04	Command	I2C 地址和 start/read/write/write_multiple/stop 控制位

地址	寄存器	功能
0x08	Data	写数据、读数据、 data_valid 和 data_last
0x0C	Prescale	I2C 分频配置

当处理器需要发起 I2C 写操作时，先写入数据寄存器或写 FIFO，再写命令寄存器发起 start/write/stop。需要读取外设时，处理器写命令寄存器发起读操作，再查询状态或数据寄存器获取返回数据。状态寄存器中的 miss_ack 可用于判断从设备是否响应，FIFO 状态位可用于避免上溢或下溢。

5.4 Wishbone 封装模块实现

Wishbone 封装模块与 AXI Lite 封装模块功能类似，但接口信号和寄存器组织方式适配 Wishbone 总线。i2c_master_wbs_8 使用 8 位数据总线，因此分频寄存器被拆分为低字节和高字节。其寄存器包括 Status、FIFO Status、Cmd Addr、Command、Data、Prescale Low 和 Prescale High。

引入 Wishbone 封装后，同一 I2C 主机核心不再局限于 AXI 生态，也可以接入采用 Wishbone 的开源 SoC 或软核处理器系统。处理器侧看到的是一组普通外设寄存器，而底层仍由 I2C 主机模块完成总线时序，因此该封装主要提升了 IP 的移植性和复用范围。

5.5 I2C 初始化模块实现

i2c_init 用于处理上电后固定外设配置这类场景。设计者可以把配置流程写入 init_data ROM，模块运行时逐项读取表项并解释为地址选择、数据写入、延时等待或 stop 等动作。对于不包含 CPU 的纯 FPGA 系统，这种方式可以让硬件在启动阶段自动完成部分 I2C 外设初始化。

模块启动后，状态机按顺序读取 ROM 表项，并将其转换为 I2C 主机命令接口和写数据接口输出。若使用单设备模式，ROM 表直接包含目标地址和写入数据；若使用多设备模式，ROM 表可先定义数据块，再定义多个设备地址，使同一初始化序列在多个设备上执行。

该模块适用于 FPGA 系统上电后自动配置外设。例如在没有 CPU 参与的系统中，可以通过修改 ROM 表，在启动阶段自动配置时钟芯片、PLL 或传感器寄存器。需要注意的是，当前源码中的 init_data 是示例数据，实际应用时必须根据目标外设的数据手册修改。

5.6 AXI Stream FIFO 实现

axis_fifo 是项目中复用性较强的流式缓冲模块。除了基本的数据宽度和深度参数外，它还可以按需启用 tkeep、tlast、tid、tdest、tuser 等 AXI Stream 附加信号，并支持不同输出流水和帧处理选项。放在 I2C 控制器封装中时，该 FIFO 主要用于暂存命令、待写数据和已读数据，缓解寄存器访问速度与 I2C 总线速度之间的差异。

FIFO 使用写指针和读指针管理存储空间。当写指针追上读指针的特定模式时，FIFO 判定为满；当写指针与读指针相等时，FIFO 判定为空。对于普通流式数据，`s_axis_tready` 在 FIFO 未满时有效；对于帧 FIFO 模式，还需要结合 `tlast` 判断帧边界。该模块可在 I2C 控制器封装中用于命令缓冲、写数据缓冲和读数据缓冲。

6 系统测试与结果分析

6.1 测试环境

根据项目 README，测试环境需要安装 MyHDL、Icarus Verilog，并保证 `myhdl.vpi` 能被仿真器加载。测试资料主要放在 `tb/` 目录，其中既有 Python 测试脚本，也有 Verilog 测试顶层。一般流程是先由脚本调用 Icarus Verilog 编译 RTL 和 testbench，生成 `.vvp` 仿真文件，再通过 `vvp -m myhdl` 启动协同仿真，使 Python 行为模型与 Verilog DUT 同步运行。

以 `test_i2c_master.py` 为例，测试脚本会在 Python 侧创建命令源、写数据源和读数据接收端，并通过 I2C 存储器行为模型模拟外部从设备。脚本中设置了两个从设备地址，其中 `0x50` 用于普通访问，`0x51` 增加了响应延迟，用来观察主机在不同从设备响应条件下的读写行为。

6.2 测试内容设计

本文根据项目测试平台和模块功能，规划以下测试内容。

6.2.1 I2C 主机写测试

主机写测试的重点是确认控制器能否按照命令完成一次规范写传输。测试平台先向命令通道提交从设备地址以及 `start`、`write`、`stop` 等控制信息，再通过写数据通道送入待发送字节。验证时需要检查 SCL/SDA 波形中是否包含起始条件、地址与写方向位、数据位、ACK 周期和停止条件，并同步观察 `busy`、`bus_active` 等状态变化。

6.2.2 I2C 主机读测试

主机读测试用于验证读事务的数据返回路径。测试前，I2C 存储器模型中需要预置若干数据；测试过程中，主机通过命令接口发送读请求并驱动地址阶段。随后应检查读方向位是否正确、返回字节是否进入 `m_axis_data_tdata`，以及最后一个字节是否由 `m_axis_data_tlast` 标出。

6.2.3 多字节写与 repeated start 测试

多字节写测试关注连续数据发送过程，尤其是 `write_multiple` 命令与 `tlast` 标志之间是否配合正确。控制器应在多个字节之间保持事务连续，并在末字节后按命令决定是否停止。Repeated start 场景则模拟寄存器型外设常见访问方式：先写入寄存器地址，再不释放总线直接切换为读操作。该测试可以反映命令层状态机对组合事务的处理能力。

6.2.4 从机地址匹配测试

从机地址匹配测试需要构造两类访问：一类使用与 `device_address` 匹配的地址，另一类使用不匹配地址。匹配时，从机应在应答周期拉低 SDA，并根据读写方向进入后续数据阶段；不匹配时，从机应保持释放状态，不应参与数据传输。若改变 `device_address_mask`，还可以进一步观察掩码匹配对地址响应范围的影响。

6.2.5 时钟拉伸测试

时钟拉伸测试主要面向从机读方向。测试平台可以故意延迟 AXI Stream 输入侧的数据，使从机在外部主机请求读取时暂时没有可发送字节。此时应观察 SCL 是否被从机保持在低电平；当数据重新有效后，SCL 应被释放，后续位传输继续进行。

6.2.6 AXI Lite 和 Wishbone 寄存器测试

寄存器测试用于验证总线封装模块的可用性。测试内容包括：读状态寄存器、写命令寄存器、写数据寄存器、读数据寄存器、配置分频寄存器、检查 FIFO 空满状态和溢出状态。对于 AXI Lite，需要验证读写地址、数据、响应通道是否符合协议；对于 Wishbone，需要验证 `cyc`、`stb`、`we`、`ack` 等信号配合是否正确。

6.3 当前验证结果

根据项目资料可以确认：

1. 项目为主要 RTL 模块提供了对应测试脚本和 Verilog 测试顶层，例如 `test_i2c_master.py/.v`、`test_i2c_slave.py/.v`、`test_i2c_master_axil.py/.v`、`test_i2c_master_wbs_8.py/.v`、`test_i2c_slave_wbm.py/.v` 等。
2. 测试平台包含多类辅助模型，不仅能够产生简单激励，还可以模拟外部 I2C 设备、AXI Stream 数据源与接收端，以及 AXI Lite、Wishbone 总线访问过程。依托这些模型，可以围绕主机核心、从机核心和总线封装模块分别搭建验证场景。
3. 项目采用 MyHDL 与 Icarus Verilog 协同仿真，可通过 Python 构造更灵活的激励与检查逻辑。

由于当前写作阶段未在本机实际执行完整仿真命令，本文不虚构仿真通过率、波形截图、资源占用或最高频率等数据。后续正式定稿时应补充如下内容：

表 6.1 待补充测试结果记录表

测试项	预期结果	实际结果	备注
I2C 主机单字节写	产生 start、地址、写数据、ACK 和 stop	待补充	需附波形
I2C 主机多字节写	多字节连续传输，末字节由 tlast 标记	待补充	需附日志
I2C 主机读	读回数据与模型数据一致	待补充	需附波形

测试项	预期结果	实际结果	备注
I2C 从机写	外部写入数据正确 输出到 AXI Stream	待补充	需附检查结果
I2C 从机读	AXI Stream 输入 数据正确发送到 I2C 总线	待补充	需验证 ACK/NACK
时钟拉伸	数据未准备好时 SCL 被拉低	待补充	需设置输入暂停
AXI Lite 寄存器 访问	读写寄存器与状态 位正确	待补充	需记录寄存器值
Wishbone 寄存器 访问	Wishbone 读写握 手正确	待补充	需记录总线波形

6.4 结果分析

从设计结构和测试平台完整性来看，该项目具备较好的可验证性。其一，主从核心均采用标准化接口，测试平台可以通过行为模型模拟外部 I2C 设备或主机，便于构造读写场景。其二，AXI Stream、AXI Lite 和 Wishbone 均有相应测试辅助模型，可降低测试代码编写难度。其三，Python 语言适合生成复杂激励和检查返回数据，因此 MyHDL 协同仿真比纯 Verilog testbench 更便于组织系统级场景。

但从毕业论文完整性角度看，仅有测试平台文件还不足以证明系统已经通过验证。正式论文应进一步补充仿真环境版本、运行命令、关键日志、波形截图和错误场景测试结果。若进行 FPGA 综合，还应补充目标芯片型号、综合工具版本、时钟约束、资源利用率和时序报告。若进行板级测试，还应补充外设型号、上拉电阻、I2C 速率、逻辑分析仪波形和读写数据记录。

7 总结与展望

7.1 全文总结

本文围绕 Verilog I2C interface 项目，对 I2C 总线控制器的结构、接口和验证方法进行了整理分析。论文首先从 FPGA 板级外设管理需求入手，说明 I2C 控制器在 EEPROM、传感器、时钟芯片和 SoC 外设扩展中的应用价值；随后介绍 I2C 协议、Verilog HDL、FPGA、AXI Stream、AXI Lite、Wishbone 以及 MyHDL/Icarus Verilog 协同仿真，为后续需求分析和模块设计提供基础。

在需求分析部分，本文从主机通信、从机通信、总线封装、初始化序列和测试验证等方面归纳了系统功能需求，并总结了可综合性、可复用性、可配置性和可验证性等非功能需求。在系统设计部分，本文将项目划分为协议核心层、流式接口层、总线封装层和验证层，明确了各层之间的数据流和控制关系。

在实现分析中，论文分别讨论了项目中的几个关键模块。`i2c_master` 负责将上层命令转换为具体 I2C 时序，`i2c_slave` 负责监听外部主机并完成地址响应和数据交换，`i2c_master_axil`、`i2c_master_wbs_8` 分别提供 AXI Lite 与 Wishbone

寄存器接口, `i2c_init` 用 ROM 表组织上电初始化流程, `axis_fifo` 则承担命令与数据缓冲作用。

在测试验证部分, 本文结合 MyHDL 与 Icarus Verilog 协同仿真平台, 设计了主机写、主机读、多字节写、从机地址匹配、时钟拉伸、AXI Lite 寄存器访问和 Wishbone 寄存器访问等测试内容。由于当前材料尚未包含本机仿真和综合结果, 论文未编造实验数据, 而是明确列出需要补充的测试记录。

7.2 创新点与特点

本文所分析设计具有以下特点:

1. 模块边界较清楚。主机、从机、FIFO、初始化逻辑以及 AXI Lite、Wishbone 封装分别独立成模块, 便于单独分析、移植和复用。
2. 接口覆盖面较广。项目同时使用 AXI Stream、AXI Lite 和 Wishbone, 使同一 I2C 核心能够适配纯逻辑控制、处理器寄存器访问和开源 SoC 集成等不同场景。
3. 主机控制层次较分明。事务级命令处理与位级物理时序分开实现, 降低了状态机理解和调试难度。
4. 从机侧考虑了时钟拉伸。当待发送数据尚未准备好时, 从机可以通过拉低 SCL 暂停传输, 避免输出无效数据。
5. 验证基础较完善。项目配套 MyHDL 行为模型和多个测试脚本, 为后续补充仿真日志和波形提供了基础。

7.3 不足与展望

本文仍存在以下不足:

1. 未补充完整本机仿真日志和波形截图, 测试结果部分仍需进一步实证化。
2. 未进行 FPGA 综合和时序分析, 缺少 LUT、FF、BRAM、最高频率等硬件资源数据。
3. 未进行板级实物测试, 无法给出真实外设访问结果和逻辑分析仪波形。
4. 中文参考文献条目仍需通过 CNKI 或学校图书馆进一步核验作者、来源、年份和页码。
5. 当前 `i2c_init` 中的初始化数据为模板示例, 尚未结合具体外设手册形成真实应用案例。

后续工作可从以下方面展开: 第一, 搭建完整 MyHDL 与 Icarus Verilog 仿真环境, 运行项目全部测试脚本并整理日志和波形; 第二, 选择目标 FPGA 平台进行综合和时序分析, 评估资源占用与最高工作频率; 第三, 选取 EEPROM、温度传感器或时钟芯片等实际 I2C 外设进行板级验证; 第四, 补充异常场景测试, 如 NACK、FIFO 溢出、总线占用、时钟拉伸和 repeated start; 第五, 结合具体 SoC 平台完善 AXI Lite 或 Wishbone 软件驱动示例。

参考文献

- [1] NXP Semiconductors. UM10204 I2C-bus specification and user man-

ual[EB/OL]. 2021. 链接:<https://www.nxp.com/docs/en/user-guide/UM10204.pdf>.
获取方式: NXP 官方公开 PDF。

[2] AMD/Xilinx. AXI IIC Bus Interface v2.1 LogiCORE IP Product Guide PG090[EB/OL]. 2021. 链接: <https://docs.amd.com/r/en-US/pg090-axi-iic>. 获取方式: AMD 官方文档。

[3] OpenCores. I2C controller core[EB/OL]. 链接:<https://opencores.org/projects/i2c>.
获取方式: OpenCores 项目页面。

[4] OpenCores. Wishbone B4 Specification[EB/OL]. 2010. 链接:https://cdn.opencores.org/downloads/wbspec_
获取方式: OpenCores 官方规范文档。

[5] Bergeron J. Writing Testbenches using SystemVerilog[M]. Boston: Springer, 2006. DOI: <https://doi.org/10.1007/0-387-25038-2>.

[6] Spear C, Tumbush G. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features[M]. New York: Springer, 2012. DOI: <https://doi.org/10.1007/978-1-4614-0715-7>.

[7] ARM Limited. AMBA AXI and ACE Protocol Specification[EB/OL]. 获取方式: ARM 官方规范文档或授权文档渠道。注: 最终定稿需补充具体版本号和链接。

[8] IEEE. IEEE Standard for Verilog Hardware Description Language: IEEE Std 1364[S]. 获取方式: IEEE Xplore 或学校图书馆授权访问。注: 最终定稿需补充具体版本和访问链接。

[9] 【待核验】基于 FPGA 的 I2C 总线接口设计 [J/学位论文]. 注: 中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。

[10] 【待核验】基于 Verilog 的 I2C 控制器设计与实现 [J/学位论文]. 注: 中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。

[11] 【待核验】I2C 总线协议的 FPGA 设计与仿真 [J/学位论文]. 注: 中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。

[12] 【待核验】AXI 总线接口控制器的设计与实现 [J/学位论文]. 注: 中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。

致谢

论文从选题、资料整理到正文撰写,都得到了指导教师的耐心指导。老师在课题方向把握、I2C 协议理解、FPGA 硬件设计方法以及论文结构安排等方面提出了许多具体建议,使本人能够逐步理清数字系统设计与验证的基本流程,也对毕业设计工作的完整性有了更深入的认识。

还要感谢开源社区对 Verilog I2C interface 项目及相关资料的公开分享。项目中的主机、从机、AXI Lite/Wishbone 封装以及 MyHDL 测试平台,为本文开展模块分析和验证方案整理提供了直接参考。与此同时,同学和朋友在资料查找、环境准备和论文修改阶段给予了许多交流和帮助,在此一并表示感谢。

最后，感谢家人在学习和毕业设计期间给予的支持与鼓励。由于本人能力和时间有限，论文中仍存在不足之处，恳请各位老师批评指正。

附录

附录 A 项目主要文件结构

```
verilog-i2c-master/  
  README.md  
  rtl/  
    axis_fifo.v  
    i2c_init.v  
    i2c_master.v  
    i2c_master_axil.v  
    i2c_master_wbs_8.v  
    i2c_master_wbs_16.v  
    i2c_single_reg.v  
    i2c_slave.v  
    i2c_slave_axil_master.v  
    i2c_slave_wbm.v  
  tb/  
    axil.py  
    axis_ep.py  
    i2c.py  
    wb.py  
    test_i2c_master.py  
    test_i2c_slave.py  
    test_i2c_master_axil.py  
    test_i2c_master_wbs_8.py  
    test_i2c_master_wbs_16.py  
    test_i2c_slave_axil_master.py  
    test_i2c_slave_wbm.py
```

附录 B 典型仿真命令

```
cd tb  
python test_i2c_master.py
```

测试脚本内部典型编译命令如下：

```
iverilog -o test_i2c_master.vvp ../rtl/i2c_master.v test_i2c_master.v  
vvp -m myhdl test_i2c_master.vvp -lxt2
```

附录 C 待补充材料清单

1. 学校模板中的封面、任务书和个人信息；
2. CNKI 已核验中文参考文献；

3. MyHDL/Icarus Verilog 仿真日志;
4. 关键 I2C 波形截图;
5. FPGA 综合资源报告;
6. 板级外设读写实验记录;
7. 最终 DOCX 或 LaTeX 模板排版结果。